SE446: Big Data Engineering

Week 9A: Spark MLlib — Machine Learning at Scale

Prof. Anis Koubaa

SE 446
Alfaisal University
https://github.com/aniskoubaa/big_data_course

Spring 2026

Instructor Notes

Welcome to Week 9A. Today we tackle one of the most important topics in the course: machine learning at scale using Spark MLlib. You already know machine learning from your previous ML course — you know how to train models, evaluate them, and tune hyperparameters using scikit-learn. The question is: what happens when your data no longer fits on a single machine? This is exactly the problem MLlib solves. We will see that the math does not change, but the execution model changes fundamentally. By the end of this session, you will be able to build a complete ML pipeline on distributed data using the same Chicago Crimes dataset you have been working with all semester.

Today's Agenda

1. From Traditional ML to Distributed ML
2. The ML Pipeline API
3. Feature Engineering
4. Model Training
5. Model Evaluation
6. Model Comparison & Hyperparameter Tuning
7. Summary

Instructor Notes

Let us look at the agenda for today. We have six main sections. First, we will understand why traditional ML breaks at scale and how distributed ML solves it. Second, we will learn the ML Pipeline API with its key abstractions: Transformer, Estimator, and Pipeline. Third, we will cover feature engineering using StringIndexer and VectorAssembler. Fourth, we will train three classification models. Fifth, we will evaluate models using the metrics you already know. Sixth, we will compare models and tune hyperparameters with CrossValidator. It is important to note that each section builds on the previous one, so follow the sequence carefully.

Learning Objectives

By the end of this session you will be able to:

1. Explain **why** traditional ML breaks at scale and how distributed ML solves it
2. Describe the ML Pipeline API: `Transformer`, `Estimator`, `Pipeline`
3. Prepare features with `StringIndexer` and `VectorAssembler`
4. Train classification models (Logistic Regression, Random Forest, GBT)
5. Evaluate with AUC-ROC, accuracy, F1, precision, recall
6. Tune hyperparameters with `CrossValidator`
7. Interpret feature importances to understand model decisions

Today's Task

Build an ML pipeline to predict whether a crime results in **arrest** using the Chicago Crimes dataset — the same dataset you've been analyzing all semester.

Instructor Notes

These seven learning objectives represent the complete ML workflow in Spark. Objectives one and two are conceptual — understanding why we need distributed ML and how the Pipeline API is structured. Objectives three through five are practical — you will write code to prepare features, train models, and evaluate results. Objectives six and seven are about optimization — tuning hyperparameters and interpreting what the model learned. It is important to note that our task today is the same Chicago Crimes dataset you have been using since Week 7. The question we are answering is: given the district, crime type, hour, and domestic flag, can we predict whether a crime leads to arrest? We will see this end-to-end in the hands-on session on Wednesday.

Outline

1. From Traditional ML to Distributed ML
2. The ML Pipeline API
3. Feature Engineering
4. Model Training
5. Model Evaluation
6. Model Comparison & Hyperparameter Tuning
7. Summary

You Already Know Machine Learning

*This is **not** your first ML course. Let's build on what you know.*

What You Already Know

- Train/test split
- Feature engineering
- Logistic Regression, Random Forest
- Accuracy, Precision, Recall, F1
- Cross-validation, GridSearch
- scikit-learn API

What's New Today

- **Why** scikit-learn breaks at scale
- **How** distributed training works
- The Spark MLlib Pipeline API
- Same algorithms, different execution
- Training on terabytes, not gigabytes
- When to use which tool

Today's Guiding Question

Same math, same intuition — but what changes when your data doesn't fit on one machine?

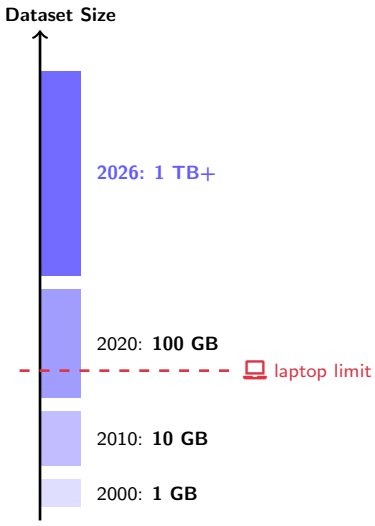
Instructor Notes

I want to be very clear about something: this is not an ML fundamentals lecture. You already know how to train models, split data, and evaluate results. On the left side of this slide you see everything you already master — train/test split, feature engineering, the scikit-learn API. What is new today is on the right side. The question is: why does scikit-learn break when data gets large? The answer involves three walls — memory, compute, and scale — which we will examine on the next slide. Today's guiding question is simple: the math stays the same, but what changes when data does not fit on one machine? Keep this question in mind throughout the entire session.

ML Is Everywhere — But Data Is Getting Bigger

Real problems that use ML on large data:

- **Crime prediction**: Can we predict whether a crime will lead to arrest? *(Today's task)*
- **Healthcare**: Detect disease from millions of patient records
- **Finance**: Fraud detection on billions of transactions per day
- **Web**: Recommendation systems for 100M+ users



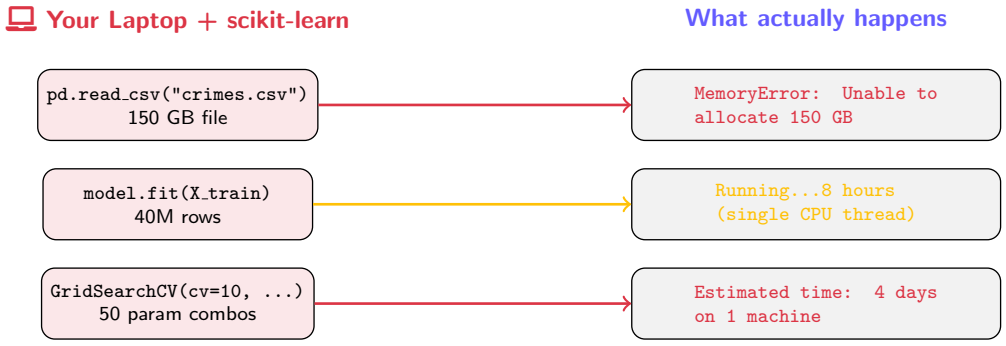
The Challenge

These datasets do **not fit on a laptop**.
A single machine crashes, stalls, or takes days.

Instructor Notes

Let us start with motivation. Look at the diagram on the right: dataset sizes have grown from one gigabyte in the year 2000 to over one terabyte today. The dashed red line shows the typical laptop limit — around 16 to 32 gigabytes of RAM. Everything above that line cannot be processed on a single machine. On the left, I give you four concrete scenarios. Crime prediction is today's task — can we predict arrest using the Chicago Crimes dataset? Healthcare, finance, and web recommendation systems are other real examples. The common thread is that these datasets simply do not fit on a laptop. As a consequence, we need a distributed approach, and that is exactly where Spark MLlib comes in.

What Breaks When You Try scikit-learn on Big Data



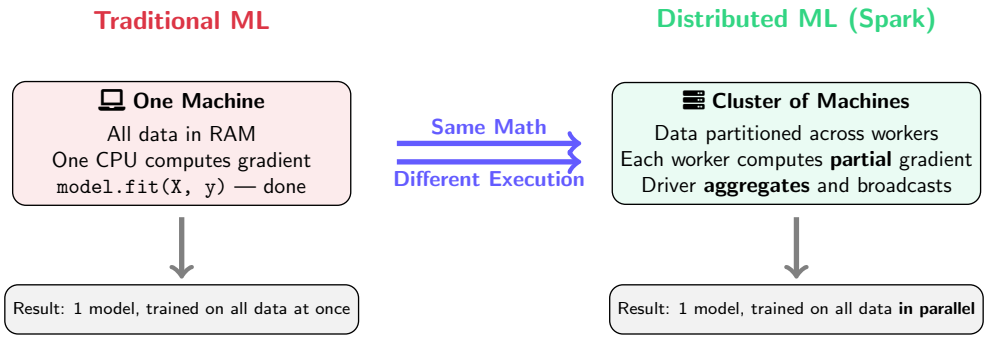
The Root Cause — Three Walls

Memory wall: scikit-learn loads **all data into one machine's RAM**.
Compute wall: Trains on **one CPU** (or a few cores at best).
Scale wall: There is **no way to add more machines** — you're stuck.

Instructor Notes

Now let us be precise about what goes wrong. There are three walls you hit with scikit-learn on big data. First, the memory wall: when you call `pd.read_csv()` on a 150 gigabyte file, pandas tries to load everything into RAM and you get a MemoryError. Second, the compute wall: even if you had enough RAM, `model.fit()` runs on a single CPU thread, so training on 40 million rows takes eight hours. Third, the scale wall: there is no way to add more machines to scikit-learn — you are stuck with whatever hardware you have. This is why we need a fundamentally different execution model. The answer is not better hardware; the answer is distributing the work across a cluster. That is exactly what Spark MLlib provides.

Traditional ML vs Distributed ML — What Actually Changes



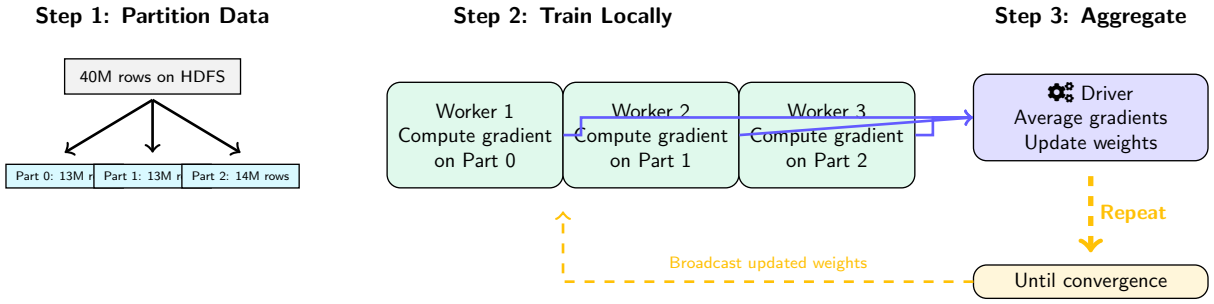
The Fundamental Shift

The **algorithm** (e.g., gradient descent) is the same. The **execution** changes: data is split, gradients are computed locally, then **aggregated** on the driver. You don't learn new math — you learn a new **execution model**.

Instructor Notes

This is the key conceptual slide of the entire lecture. On the left you see traditional ML: one machine, all data in RAM, one CPU computes the gradient, done. On the right you see distributed ML with Spark: the data is partitioned across workers, each worker computes a partial gradient on its local partition, and the driver aggregates these partial results. The critical insight is in the middle: same math, different execution. You are not learning a new algorithm today. Gradient descent is still gradient descent. What changes is that instead of one machine doing all the work, multiple machines share the load. As such, you get near-linear speedup by adding more workers. This is the fundamental shift from traditional to distributed ML.

How Spark Actually Trains a Model



Why This Is Faster

3 workers each process 13M rows = **3**× **faster** than 1 machine on 40M rows. Add more workers → near-linear speedup. **Same math, parallel execution.**

Instructor Notes

Let us now walk through the mechanics step by step. There are three steps in each training iteration. Step one: the 40 million rows sitting on HDFS are partitioned across three workers, roughly 13 million rows each. Step two: each worker independently computes the gradient on its local partition. This is the key — no worker needs to see the entire dataset. Step three: the driver collects the partial gradients from all workers, averages them, updates the model weights, and broadcasts the new weights back. This cycle repeats until convergence. The result is that three workers processing 13 million rows each is approximately three times faster than one machine processing 40 million rows. We will see this pattern again when we discuss CrossValidator later.

Spark MLlib — Same Idea, Scales Across the Cluster

scikit-learn (1 machine)

```
import pandas as pd
from sklearn.ensemble import (
    RandomForestClassifier)
from sklearn.model_selection import (
    train_test_split)

df = pd.read_csv("crimes.csv")
X = df[["District","Hour"]]
y = df["Arrest"]
X_tr, X_te, y_tr, y_te = (
    train_test_split(X, y, test_size=0.2))

model = RandomForestClassifier()
model.fit(X_tr, y_tr)
```

✗ Crashes on large data

Spark MLlib (cluster)

```
from pyspark.ml.classification import (
    RandomForestClassifier)
from pyspark.ml import Pipeline

df = spark.read.csv("hdfs:///...")
train_df, test_df = (
    df.randomSplit([0.8, 0.2]))

rf = RandomForestClassifier(
    featuresCol="features",
    labelCol="label")
pipeline = Pipeline(
    stages=[indexer, assembler, rf])

model = pipeline.fit(train_df)
```

✓ Runs on terabytes, any cluster

The Key Difference

scikit-learn: data is a numpy array in RAM → one machine does everything.
MLlib: data is a distributed DataFrame on HDFS → cluster works in parallel.

Instructor Notes

Now let us see what this looks like in actual code. On the left you have the scikit-learn version you already know: load with pandas, select columns, split, fit. On the right you have the Spark MLlib version: load from HDFS, split with `randomSplit`, define a pipeline with stages, and fit. Notice how similar the structure is. The key difference is not the API — it is where the data lives. In scikit-learn, data is a numpy array in RAM on one machine. In MLlib, data is a distributed DataFrame spread across HDFS. As a consequence, the Spark version can handle terabytes because it never tries to load everything into one machine's memory. We will build this pipeline step by step throughout the rest of the lecture.

Your scikit-learn Knowledge → MLlib Translation Guide

Concept	scikit-learn	Spark MLlib
Load data	`pd.read_csv()`	`spark.read.csv()`
Data structure	`DataFrame` (pandas)	`DataFrame` (Spark)
Feature vector	numpy array / column list	`VectorAssembler`
Encode categories	`LabelEncoder`	`StringIndexer`
One-hot encoding	`OneHotEncoder`	`OneHotEncoder`
Train/test split	`train_test_split()`	`df.randomSplit()`
Chain steps	`sklearn.pipeline`	`pyspark.ml.Pipeline`
Train	`model.fit(X, y)`	`pipeline.fit(df)`
Predict	`model.predict(X)`	`model.transform(df)`
Grid search	`GridSearchCV`	`CrossValidator`
Save model	`joblib.dump()`	`model.save("hdfs://...")`

Notice the Pattern

The **concepts are identical**. The API names are slightly different, and the data lives on a **cluster instead of RAM**.

Instructor Notes

This table is your cheat sheet for the entire lecture. I recommend you keep it visible as we go through the code. Every row maps a concept you already know from scikit-learn to its MLlib equivalent. For example, `LabelEncoder` becomes `StringIndexer`. `train_test_split` becomes `randomSplit`. `GridSearchCV` becomes `CrossValidator`. There are two important differences to note. First, in MLlib you do not pass X and y separately — you pass a single DataFrame with a features column and a label column. Second, prediction uses `model.transform()` instead of `model.predict()`, because in Spark the model adds a new column to the DataFrame rather than returning a separate array. We will see both of these patterns in action shortly.

When to Use scikit-learn vs Spark MLlib

Use scikit-learn

- ✔ Data fits in RAM (<5–10 GB)
- ✔ Rapid prototyping / exploration
- ✔ Need exotic algorithms (XGBoost, SVM kernels, DBSCAN, etc.)
- ✔ Small team, quick iteration

Use Spark MLlib

- ✔ Data is too large for one machine
- ✔ Data already lives on HDFS/S3
- ✔ Need to train on 10 GB+ regularly
- ✔ Production pipeline at scale
- ✔ Need parallel hyperparameter tuning

Decision: data size + infrastructure

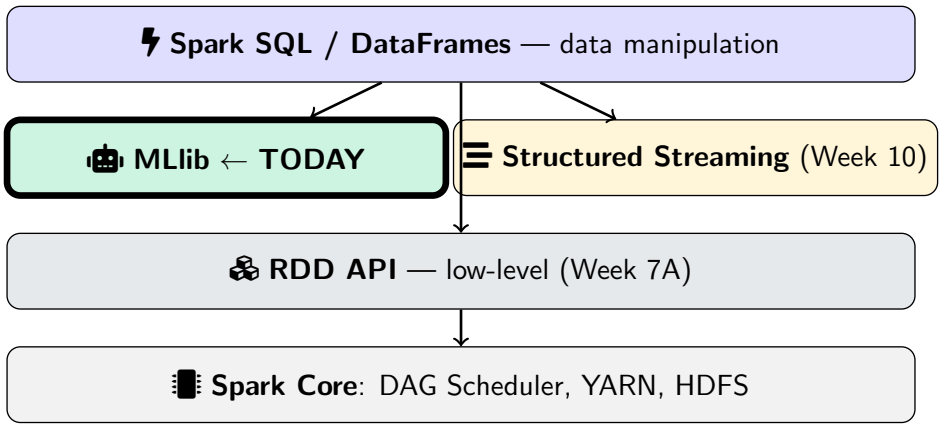
Common Mistake

Distributed $\neq$ always better. Spark has **overhead** (serialization, shuffle, coordination). On small data, scikit-learn is **faster**. Use the right tool for the right scale.

Instructor Notes

This is a question I get every semester: should I always use Spark MLlib? The answer is no. Distributed computing has overhead — serialization, network shuffles, task scheduling. On small data that fits in RAM, scikit-learn is actually faster because it avoids all that overhead. The decision comes down to two factors: data size and infrastructure. If your data is under five to ten gigabytes and fits in memory, use scikit-learn. If your data is larger, or if it already lives on HDFS or S3, use MLlib. It is also important to note that scikit-learn has a much richer algorithm library — things like DBSCAN, kernel SVMs, and XGBoost are not available in MLlib. So use the right tool for the right scale.

MLlib in the Spark Ecosystem



API Warning

`pyspark.mllib` (RDD-based) — **old, maintenance mode, do NOT use**.
`pyspark.ml` (DataFrame-based) — **current, recommended**. We use this.

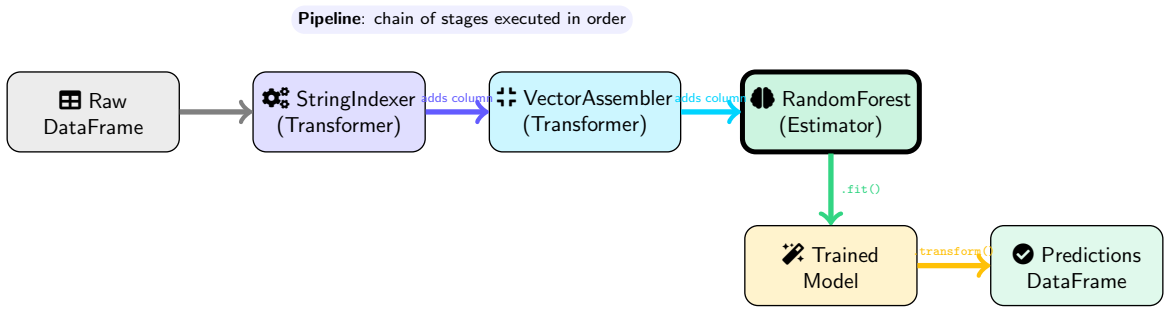
Instructor Notes

Let us situate MLlib within the Spark ecosystem you already know. At the bottom is Spark Core with the DAG scheduler, YARN integration, and HDFS. Above that is the RDD API from Week 7A. Above that are the high-level libraries: Spark SQL and DataFrames for data manipulation, MLlib for machine learning, and Structured Streaming which we will cover in Week 10. MLlib sits on top of DataFrames, which means everything you learned about Spark SQL — selecting columns, filtering, grouping — applies directly when preparing data for ML. There is one critical warning: there are two MLlib packages. The old one is `pyspark.mllib`, which is RDD-based and in maintenance mode. The new one is `pyspark.ml`, which is DataFrame-based. We use only `pyspark.ml`. Do not confuse the two.

Outline

1. From Traditional ML to Distributed ML
2. The ML Pipeline API
3. Feature Engineering
4. Model Training
5. Model Evaluation
6. Model Comparison & Hyperparameter Tuning
7. Summary

Pipeline Concept — Assembly Line



Transformer

Takes a DF, returns a DF
Method: `.transform(df)`
Example: VectorAssembler

Estimator

Takes a DF, **learns** params
Method: `.fit(df)` → Model
Example: RandomForest

Instructor Notes

Think of a pipeline as an assembly line in a factory. Raw material goes in on the left, passes through several stations, and a finished product comes out on the right. In our case, the raw material is a DataFrame, and the finished product is predictions. There are two types of stages. A Transformer takes a DataFrame and returns a new DataFrame with an added column — it uses `.transform()`. An Estimator takes a DataFrame, learns parameters from it using `.fit()`, and produces a Model, which is itself a Transformer. This is the same pattern as scikit-learn's fit-then-transform, but formalized into a pipeline abstraction. The pipeline executes all stages in order, which guarantees that train and test data go through exactly the same transformations.

Transformer vs Estimator — Detailed Comparison

	Transformer	**Estimator**
Method	`.transform(df)` → df	`.fit(df)` → Model (Transformer)
Parameters	Fixed (no learning)	Learned from data
Examples	VectorAssembler, OneHotEncoder (fitted), StringIndexer (fitted)	LogisticRegression, RandomForest, StringIndexer (unfitted)
sklearn analogy	`.transform()` after `.fit()`	`.fit()` itself
Analogy	A **ruler** (measures directly)	A **student** (learns, then measures)

Important Nuance

`StringIndexer` starts as an **Estimator** (learns category → index mapping via `.fit()`), then produces a fitted **Transformer** (applies the mapping via `.transform()`). Same as `sklearn.preprocessing.LabelEncoder` — `.fit()` then `.transform()`.

Instructor Notes

Let us clarify the distinction between Transformer and Estimator because students often confuse them. A Transformer has fixed parameters — it does not learn anything. Think of a ruler: it measures directly without needing to study the data first. A VectorAssembler is a pure Transformer. An Estimator, on the other hand, must learn from data before it can act. Think of a student who studies first, then takes the exam. The important nuance is that some components start as Estimators and become Transformers after fitting. `StringIndexer` is the best example: when you create it, it is an Estimator because it needs to learn which categories exist. After calling `.fit()`, it produces a fitted Transformer that can map strings to indices. This is exactly like scikit-learn's `LabelEncoder` with its fit-then-transform pattern.

Why Pipelines Matter **More** in Distributed ML

✗ Without Pipeline (danger zone)

- Fit StringIndexer on full data → **data leakage**
- Manually apply each step → **inconsistent** train vs test
- Can't serialize to HDFS → **can't deploy** to cluster
- Each worker needs transforms → **manual coordination**

✓ With Pipeline

- `pipeline.fit(train_df)` → fits **only on train**
- `model.transform(test_df)` → **same steps** automatically
- `model.save("hdfs://...")` → **any worker** can load
- Pipeline ships transforms to every worker **automatically**

```
pipeline = Pipeline(stages=[
    string_indexer,        # Stage 1: encode categories
    vector_assembler,      # Stage 2: assemble features
    classifier             # Stage 3: train model
])
model = pipeline.fit(train_df)       # fit everything
predictions = model.transform(test_df) # apply everything
```

In scikit-learn you *can* skip pipelines. In Spark, you shouldn't.

Distributed data across nodes means transforms must travel **with** the model.

Instructor Notes

The question is: why do pipelines matter more in distributed ML than in scikit-learn? The answer has four parts. Without a pipeline, you risk data leakage by fitting transformers on the full dataset before splitting. You risk inconsistency by applying different steps to train and test. You cannot serialize individual steps to HDFS for deployment. And you must manually coordinate transformations across workers. With a pipeline, all of these problems disappear. `pipeline.fit(train_df)` fits every stage only on training data. `model.transform(test_df)` applies the exact same steps. And you can save the entire fitted pipeline to HDFS so any worker in the cluster can load it. This is why in scikit-learn you can get away without pipelines, but in Spark you really should not.

Without a Pipeline — The Code You'd Have to Write

What does it actually look like when you skip the Pipeline and do everything manually?

```
# WRONG: fit on full data BEFORE splitting => data leakage!
indexer = StringIndexer(inputCol="Primary Type",
                        outputCol="crime_index")
indexer_model = indexer.fit(df)        # sees test data too!
df = indexer_model.transform(df)

assembler = VectorAssembler(
    inputCols=["District","crime_index","Hour"],
    outputCol="features")
df = assembler.transform(df)

train_df, test_df = df.randomSplit([0.8, 0.2])

rf = RandomForestClassifier(featuresCol="features",
                            labelCol="label")
model = rf.fit(train_df)
predictions = model.transform(test_df)

# Now you need to deploy...
# Save the indexer? the assembler? the model? separately?
indexer_model.save("hdfs:///models/indexer")  # manual
model.save("hdfs:///models/rf_model")         # manual
# Every worker must load BOTH in the right order!
```

Three Problems Here

1. StringIndexer fitted on full data → **data leakage** (test info leaks into train). **2.** Must save/load each stage separately → **error-prone deployment**. **3.** Workers must manually coordinate transform order → **fragile at scale**.

Instructor Notes

Now I want to show you what happens if you skip the Pipeline and try to do everything manually. Look at this code carefully. The first red flag is on line three: we call indexer.fit on the full dataframe, before splitting into train and test. That means the indexer has seen the test data — that is textbook data leakage. The second problem is deployment. When you want to save this model to HDFS, you have to save the indexer model, the assembler configuration, and the random forest model all separately. Then every worker that needs to make predictions must load all three in the right order. If someone changes the order or forgets a step, the predictions are silently wrong. With a Pipeline, you call pipeline.fit once, model.save once, and PipelineModel.load once. Everything stays bundled together. This is why I said earlier: in scikit-learn you can get away without pipelines because everything runs on one machine. In Spark, skipping pipelines creates real operational risk.

Outline

1. From Traditional ML to Distributed ML
2. The ML Pipeline API
3. Feature Engineering
4. Model Training
5. Model Evaluation
6. Model Comparison & Hyperparameter Tuning
7. Summary

Our Dataset: Chicago Crimes

Column	Type	Role in ML
District	int	✔ Feature (numeric)
Primary Type	string	✔ Feature (categorical → encode)
Hour (extracted)	int	✔ Feature (numeric)
Domestic	boolean	✔ Feature (categorical → encode)
Arrest	**boolean**	🏁 **Label (what we predict)**

🎯 **Goal**: Given District, Crime Type, Hour, Domestic → predict **Arrest** (True/False)

Instructor Notes

Let us define our task precisely. We are using the Chicago Crimes dataset that you already know from Weeks 7 and 8. We have four features: District is numeric, Primary Type is a categorical string that we need to encode, Hour is numeric and extracted from the Date column, and Domestic is a boolean that we also encode. The label — what we are predicting — is Arrest: true or false. This is a binary classification problem. It is important to note that two of our features are already numeric and two require encoding. This is why we need StringIndexer before we can feed data into any ML algorithm. The goal is straightforward: given district, crime type, hour, and domestic flag, predict whether an arrest will be made.

Why These Features? Think Before You Code

In your ML course you learned: feature selection should be driven by **domain knowledge**, *not guessing.*

Feature	Domain Intuition	Expected Effect
Primary Type	NARCOTICS → almost always arrest. THEFT → rarely catch in act.	Strong predictor — crime type determines police procedure
Domestic	Many jurisdictions have **mandatory arrest** policies for domestic crimes	Strong predictor — policy-driven
District	Some districts have more resources, higher patrol density	Moderate predictor — varies by staffing
Hour	Crimes at 2 AM: fewer witnesses, harder to catch. vs 2 PM: more police visible	Moderate predictor — time affects opportunity

What We're NOT Using (and Why)

Block (address) → too many unique values, would overfit. **Date** → raw string, but we extract **Hour** as a useful signal. **Year** → could capture trend but risks data leakage on temporal data.

Instructor Notes

Before writing any code, we need to think about why we chose these features. This is domain reasoning, and it is something you learned in your ML course. There are two strong predictors. Primary Type is strong because crime type directly determines police procedure — narcotics crimes almost always lead to arrest, while theft rarely does. Domestic is strong because many jurisdictions have mandatory arrest policies for domestic crimes. There are two moderate predictors. District varies by staffing and patrol density. Hour affects opportunity — crimes at 2 AM have fewer witnesses. Equally important is what we are not using. Block has too many unique values and would cause overfitting. Year risks temporal data leakage. We will validate these intuitions later when we examine feature importances after training.

Step 1: Extract Features from Raw Data

In scikit-learn: `df["Hour"] = pd.to_datetime(df["Date"]).dt.hour`. In Spark:

```
from pyspark.sql.functions import hour, to_timestamp, col

# Load dataset from HDFS (distributed!)
df = spark.read.csv("hdfs:///data/chicago_crimes.csv",
                    header=True, inferSchema=True)

# Extract hour of day from Date string
df = df.withColumn("Hour",
    hour(to_timestamp(col("Date"),
         "MM/dd/yyyy hh:mm:ss a")))

# Select relevant columns, drop nulls
df = df.select("District", "Primary Type",
               "Hour", "Domestic", "Arrest").dropna()

# Cast label to integer (True=1, False=0)
df = df.withColumn("label",
    col("Arrest").cast("integer"))
```

Tip

Always check for nulls with `df.filter(col("Hour").isNull()).count()` before training. Nulls silently break ML models.

Instructor Notes

Now let us start coding. Step one is data preparation, which you would also do in scikit-learn. We load the CSV from HDFS — notice this is distributed, not local. We then extract the hour from the Date string using `to_timestamp` and `hour`. This is the Spark equivalent of pandas' `pd.to_datetime().dt.hour`. We select only the columns we need and drop rows with null values. Finally, we cast the Arrest boolean to an integer because MLlib expects numeric labels: one for arrest, zero for no arrest. It is important to note the `dropna()` call — nulls silently break ML models in Spark just as they do in scikit-learn. Always check for nulls before training. We will see this in detail in the hands-on session on Wednesday.

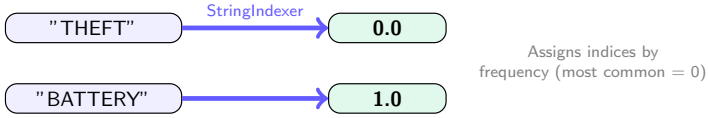
Step 2: StringIndexer — Encoding Categories

In scikit-learn: `LabelEncoder().fit_transform(df["Primary Type"])`. In Spark:

```python
from pyspark.ml.feature import StringIndexer

# "THEFT" -> 0.0, "BATTERY" -> 1.0, ...
crime_indexer = StringIndexer(
    inputCol="Primary Type",
    outputCol="crime_index",
    handleInvalid="skip"    # drop rows with unseen categories
)

# "false" -> 0.0, "true" -> 1.0
domestic_indexer = StringIndexer(
    inputCol="Domestic",
    outputCol="domestic_index",
    handleInvalid="skip"
)
```



Instructor Notes

Step two is encoding categorical columns using `StringIndexer`, which is the MLlib equivalent of scikit-learn's `LabelEncoder`. We create two indexers: one for Primary Type and one for Domestic. Each indexer specifies an input column and an output column. The `handleInvalid="skip"` parameter tells Spark to drop rows that contain categories not seen during training — this prevents errors at prediction time. There is one important detail: StringIndexer assigns indices by frequency. The most common category gets index 0, the next most common gets 1, and so on. So THEFT, being the most frequent crime type, gets 0.0. BATTERY gets 1.0. This ordering is deterministic, which means your results are reproducible across runs.

StringIndexer vs OneHotEncoder — When to Use Which

StringIndexer Only

"THEFT" → **0.0**
"BATTERY" → **1.0**
"ROBBERY" → **2.0**

StringIndexer + OneHotEncoder

"THEFT" → **(1, 0, 0)**
"BATTERY" → **(0, 1, 0)**
"ROBBERY" → **(0, 0, 1)**

	StringIndexer only	**+ OneHotEncoder**
Use with	✔ Tree models (RF, GBT)	✔ Linear models (LR)
Why?	Trees split on values, no order assumed	LR treats 2.0 as "more" than 1.0 → wrong
Feature count	1 column per category	$N - 1$ columns (sparse)

Rule of Thumb (Same as scikit-learn!)

Tree-based models → StringIndexer is enough. **Linear models** → add OneHotEncoder to avoid false ordinal relationships.

Instructor Notes

This is a concept you already know from scikit-learn, but it is worth revisiting because students often get it wrong. The question is: when do you need OneHotEncoder on top of StringIndexer? The answer depends on the model type. Tree-based models like Random Forest and GBT split on individual values, so they do not assume any ordering. For trees, StringIndexer alone is sufficient. Linear models like Logistic Regression, however, treat the numeric index as a continuous value — they would interpret 2.0 as being "greater" than 1.0, which creates a false ordinal relationship. For linear models, you need OneHotEncoder to create binary indicator columns. The rule of thumb is identical to scikit-learn: trees are fine with label encoding, linear models need one-hot encoding. We will use StringIndexer only for today since we start with tree-based models.

Step 3: VectorAssembler — Combine into Feature Vector

In scikit-learn: `X = df[["District","crime_index","Hour","domestic_index"]]`. In Spark:

```
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
    inputCols=["District", "crime_index",
               "Hour", "domestic_index"],
    outputCol="features"
)
```

District	crime_index	Hour	domestic_idx		features
8	0.0	14	1.0	→	[8, 0, 14, 1]
11	1.0	22	0.0		[11, 1, 22, 0]

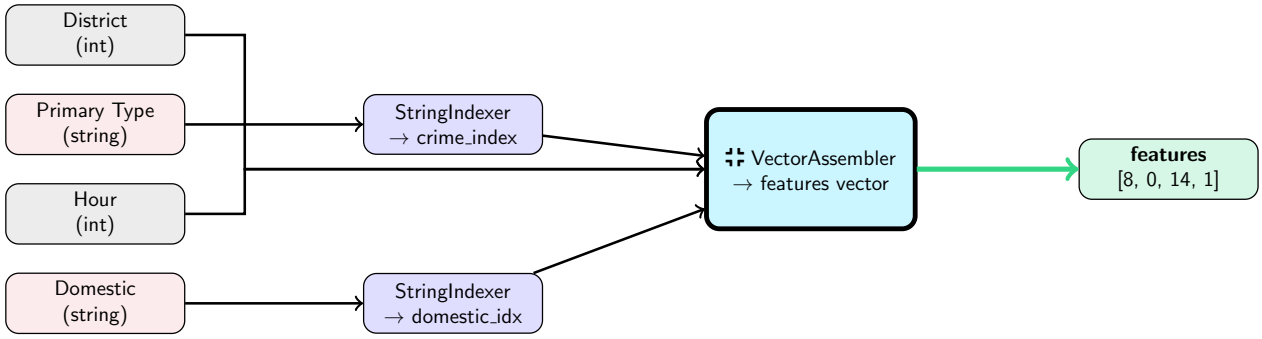
Why a Single Vector?

All Spark ML algorithms require input as a **single vector column** called `features`. This is different from scikit-learn where you pass a matrix. VectorAssembler creates a `DenseVector` per row.

Instructor Notes

Step three is assembling all feature columns into a single vector using `VectorAssembler`. This is a key difference from scikit-learn. In scikit-learn, you pass a matrix of columns directly to the model. In Spark MLlib, every algorithm expects a single column called `features` that contains a dense vector. The VectorAssembler takes multiple input columns — District, crime_index, Hour, domestic_index — and combines them into one vector per row. So the row with District 8, crime_index 0.0, Hour 14, and domestic_index 1.0 becomes the vector [8, 0, 14, 1]. It is important to note that the order of columns in `inputCols` matters — it determines the position of each value in the vector, which is critical when you later interpret feature importances.

Feature Engineering Summary



Pipeline Order Matters

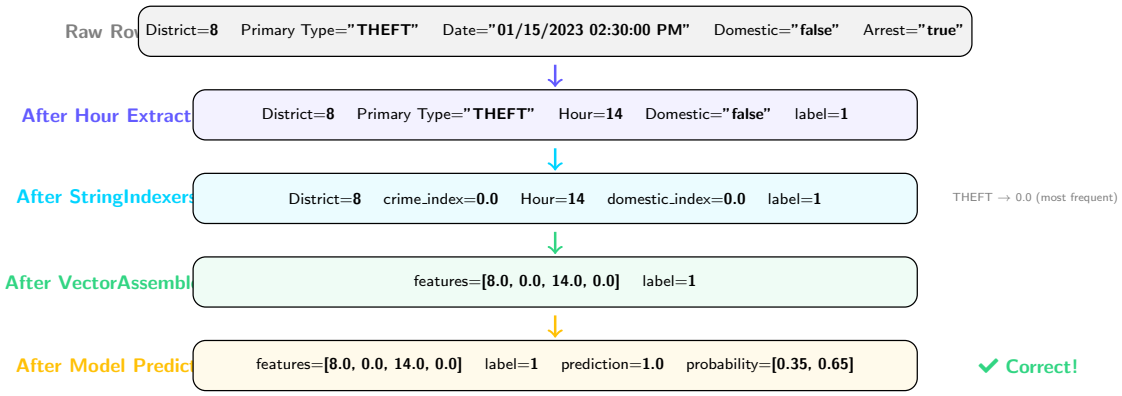
StringIndexers must run **before** VectorAssembler, because the assembler needs the numeric index columns to exist. Same logic as scikit-learn's `ColumnTransformer` ordering.

Instructor Notes

This diagram summarizes the entire feature engineering flow. On the left we have four raw columns. The two string columns — Primary Type and Domestic — pass through StringIndexers to become numeric indices. The two numeric columns — District and Hour — go directly to the VectorAssembler. The VectorAssembler combines all four numeric values into a single features vector. It is important to note that pipeline order matters. The StringIndexers must run before the VectorAssembler because the assembler needs the numeric index columns to exist. If you put the assembler first, it would fail because crime_index and domestic_index do not exist yet. This is the same ordering logic as scikit-learn's ColumnTransformer.

One Row Through the Pipeline — Step by Step

Let's trace a single Chicago crime record through every stage:



Every Row Goes Through This

The pipeline applies these 4 stages to **every row in parallel across the cluster**. On 40M rows with 4 workers, each worker processes ∼10M rows simultaneously.

Instructor Notes

Let us trace one concrete row through every stage so you can see exactly what happens. We start with a raw crime record: District 8, Primary Type THEFT, Date January 15 at 2:30 PM, Domestic false, Arrest true. After hour extraction, the Date becomes Hour 14 and Arrest becomes label 1. After StringIndexers, THEFT maps to 0.0 because it is the most frequent crime type, and false maps to 0.0 for domestic. After VectorAssembler, we get the vector [8.0, 0.0, 14.0, 0.0] with label 1. After the model predicts, we get prediction 1.0 with probability [0.35, 0.65], meaning 65 percent confidence of arrest. The prediction matches the label, so this is a correct prediction. Every row in the dataset goes through this exact sequence, in parallel across the cluster.

Outline

1. From Traditional ML to Distributed ML
2. The ML Pipeline API
3. Feature Engineering
4. Model Training
5. Model Evaluation
6. Model Comparison & Hyperparameter Tuning
7. Summary

Train/Test Split

In scikit-learn: `train_test_split(X, y, test_size=0.2)`. In Spark:

```
# 80% train, 20% test
train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)

print(f"Training: {train_df.count()} rows")
print(f"Testing:  {test_df.count()} rows")

# IMPORTANT: Check class balance!
train_df.groupBy("label").count().show()
```

Class Imbalance Warning

If 85% of crimes have `Arrest=False`, a model can just predict False and get 85% accuracy! $\rightarrow$ Always check F1 and AUC, not just accuracy.

`seed=42`

Fixed seed ensures **reproducible** splits — same rows in train/test every time you run. Same concept as `random_state=42` in sklearn.

Instructor Notes

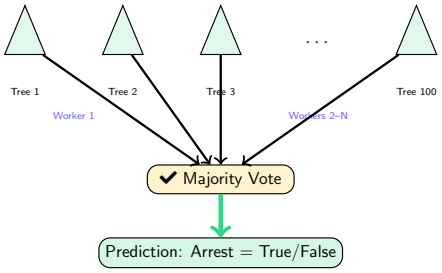
Before training any model, we split the data into 80 percent training and 20 percent testing using `randomSplit`. This is the Spark equivalent of scikit-learn's `train_test_split`. The `seed=42` parameter ensures reproducibility — you get the same split every time, just like `random_state` in scikit-learn. Now here is the critical warning: always check class balance after splitting. In the Chicago Crimes dataset, approximately 85 percent of crimes do not result in arrest. As a consequence, a naive model that always predicts "no arrest" would achieve 85 percent accuracy while catching zero criminals. This is why we need to evaluate with F1 and AUC, not just accuracy. We will return to this point in the evaluation section.

Random Forest Classifier

Same algorithm you know from your ML course — but Spark trains trees **in parallel across workers**.

```
from pyspark.ml.classification import RandomForestClassifier

rf = RandomForestClassifier(
    featuresCol="features",
    labelCol="label",
    numTrees=100,      # 100 decision trees
    maxDepth=5,        # max depth per tree
    seed=42
)
```



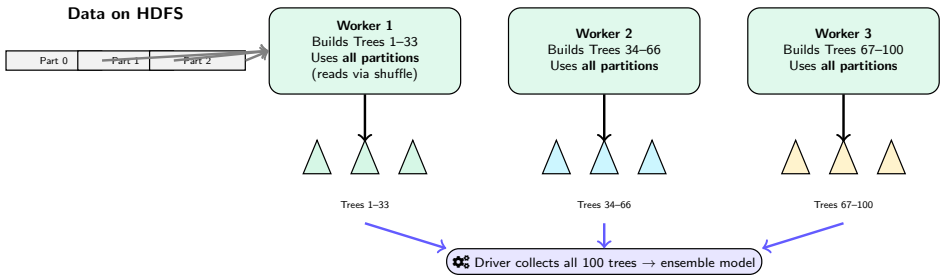
Spark distributes tree training across workers — each worker builds subset of trees in parallel

Instructor Notes

Random Forest is the same algorithm you studied in your ML course — an ensemble of decision trees where each tree votes and the majority wins. What changes in Spark is the execution. Instead of building all 100 trees sequentially on one CPU, Spark distributes tree construction across workers. Worker 1 might build trees 1 through 25, Worker 2 builds trees 26 through 50, and so on. The API requires two mandatory parameters: `featuresCol` and `labelCol`, which tell Spark which columns contain the feature vector and the label. We set `numTrees=100` for 100 trees and `maxDepth=5` to prevent overfitting. Random Forest is an excellent starting model because it handles non-linear relationships, is robust to overfitting, and parallelizes very well across the cluster.

How Spark Parallelizes Random Forest — What Runs Where?

Students often ask: does each worker build a full tree? Or each partition?



Common Misconception

✗ "Each partition builds one tree"

✓ Each **worker** builds a **subset of trees**, but each tree sees **all the data** (via bootstrap sampling across partitions).

Train/Test on a Cluster

`randomSplit([0.8,0.2])` assigns each **row** (not partition!) to train or test using a hash. Both splits remain distributed across all nodes.

Instructor Notes

This is a question that comes up every time I teach this: how exactly does Spark parallelize Random Forest? The common misconception is that each partition builds one tree. That is wrong. Here is what actually happens. Spark distributes the trees across workers, not the data. If you have 100 trees and 3 workers, Worker 1 might build trees 1 through 33, Worker 2 builds trees 34 through 66, and Worker 3 builds trees 67 through 100. Each tree needs to see all the training data to build its bootstrap sample. So data is shuffled to workers as needed. The key parallelism is at the tree level, not the partition level. Now, within a single tree, Spark also parallelizes node splits. When deciding the best split at each node, Spark computes split statistics across all partitions in parallel, then the driver picks the best split. This is a two-level parallelism: trees across workers, and node statistics across partitions.

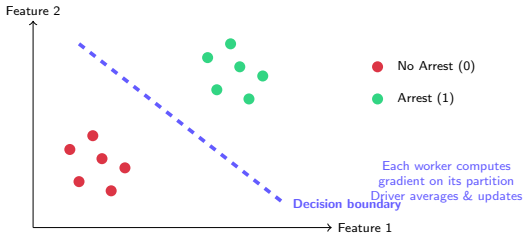
For the train-test split question: randomSplit does not split by partition. It uses a hash function on each row to decide if that row goes to train or test. So both the train set and the test set are distributed across all partitions and all nodes. This means every node has some train rows and some test rows. The 80-20 split is approximate because it is hash-based, not exact counting.

Logistic Regression

Same sigmoid function — but gradient descent runs **across workers**, *each computing partial gradients.*

```python
from pyspark.ml.classification import LogisticRegression

lr = LogisticRegression(
    featuresCol="features",
    labelCol="label",
    maxIter=100,       # max gradient descent steps
    regParam=0.01      # L2 regularization strength
)
```



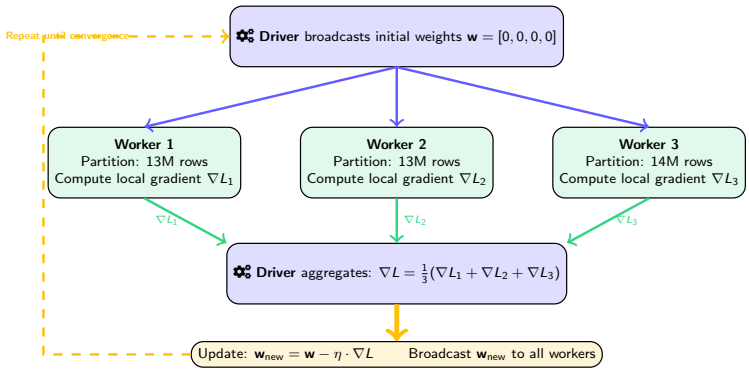
When to Use

Fast baseline model. Works best for linearly separable data. Always start here, then try Random Forest or GBT.

Instructor Notes

Logistic Regression is the same sigmoid-based classifier you know, but in Spark the gradient descent runs across workers. Each worker computes the gradient on its local partition, then the driver averages these partial gradients and updates the weights. We set `maxIter=100` for up to 100 gradient descent iterations and `regParam=0.01` for L2 regularization to prevent overfitting. The question is: when should you use Logistic Regression? The answer is: always start with it. It is the fastest model to train and gives you a baseline. If Random Forest or GBT significantly outperform it, that tells you the relationship in your data is non-linear. If Logistic Regression performs similarly, keep it because it is simpler and more interpretable. We will compare all three models later in the lecture.

Distributed Gradient Descent — How Spark Trains Logistic Regression



Key Insight

Each worker computes a gradient on its **local partition only**. The driver averages all gradients and updates the global weights. Workers never see each other's data — only gradient vectors are shipped (small!).

Instructor Notes

Let me now show you exactly how Spark distributes gradient descent for logistic regression. This is the architecture diagram you need to understand. The driver starts by broadcasting the initial weight vector to all workers. Each worker then computes a gradient using only its local partition of the data. This is the key point: Worker 1 computes the gradient on its 13 million rows, Worker 2 on its 13 million rows, and Worker 3 on its 14 million rows. None of them need to see the other workers' data.

Once all workers finish, they send their local gradient vectors back to the driver. The driver averages these gradients to get the global gradient, then updates the weight vector using the standard gradient descent update rule: w new equals w minus learning rate times gradient. The driver then broadcasts the updated weights back to all workers, and the process repeats until convergence.

The critical insight is what travels over the network: only the weight vector and gradient vectors. For our 4-feature model, that is just 4 floating point numbers each way. The actual data — 40 million rows — never moves. This is why distributed gradient descent scales: the communication cost is proportional to the number of features, not the number of rows.

Distributed Gradient Descent — Numerical Example

Let's trace one iteration with real numbers. 2 features, 2 workers, learning rate $\eta = 0.1$.

Step	Worker 1 (Part. 0)	Worker 2 (Part. 1)
Data (2 rows each)	$x_1 = [1, 2],\ y_1 = 1$ $x_2 = [2, 1],\ y_2 = 0$	$x_3 = [3, 1],\ y_3 = 1$ $x_4 = [1, 3],\ y_4 = 0$
Receive weights	$\mathbf{w} = [0.5,\ 0.3]$	
Compute $\hat{y} = \sigma(\mathbf{w} \cdot \mathbf{x})$	$\hat{y}_1 = \sigma(1.1) = 0.75$ $\hat{y}_2 = \sigma(1.3) = 0.79$	$\hat{y}_3 = \sigma(1.8) = 0.86$ $\hat{y}_4 = \sigma(1.4) = 0.80$
Local gradient $\nabla L_k = \frac{1}{n_k}\sum(\hat{y} - y)\mathbf{x}$	$\nabla L_1 = [-0.045,\ 0.145]$	$\nabla L_2 = [0.170,\ 0.330]$

Driver Aggregates and Updates

$\nabla L = \frac{1}{2}(\nabla L_1 + \nabla L_2) = \frac{1}{2}([-0.045, 0.145] + [0.170, 0.330]) = [\mathbf{0.0625},\ \mathbf{0.2375}]$

$\mathbf{w}_{\text{new}} = [0.5, 0.3] - 0.1 \times [0.0625, 0.2375] = [\mathbf{0.494},\ \mathbf{0.276}]$

Broadcast $\mathbf{w}_{new}$ to all workers → repeat for next iteration.

What Travels Over the Network?

Only the weight vector **w** (4 floats) and gradient vectors — **not** the data. Data stays local. This is why distributed gradient descent scales.

Instructor Notes

Now let us trace through one iteration with actual numbers so you can see the math. We have two workers, each with two data points, and two features. The current weights are 0.5 and 0.3.

Worker 1 has data points x1 equals 1 comma 2 with label 1, and x2 equals 2 comma 1 with label 0. It computes the dot product w dot x for each point, passes it through the sigmoid function, and gets predictions 0.75 and 0.79. Then it computes the gradient as the average of (prediction minus label) times x for each point. The result is the local gradient negative 0.045 comma 0.145.

Worker 2 does the same thing independently on its own two data points and gets a local gradient of 0.170 comma 0.330.

The driver then averages these two gradients: 0.0625 comma 0.2375. It updates the weights using the learning rate of 0.1: the new weights are 0.494 comma 0.276. These updated weights are broadcast back to all workers and the process repeats.

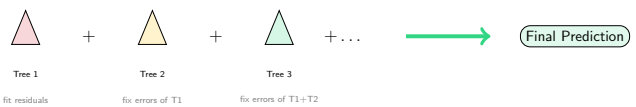
Notice what went over the network: just two numbers from each worker to the driver, and two numbers from the driver back to each worker. The four data points never moved. In a real cluster with 40 million rows and 4 features, you would still only ship 4 floats per worker per iteration. That is the fundamental reason distributed gradient descent works at scale.

Gradient-Boosted Trees (GBT) — Often the Best Performer

You may know XGBoost from your ML course. GBTClassifier is Spark's distributed equivalent.

```
from pyspark.ml.classification import GBTClassifier

gbt = GBTClassifier(
    featuresCol="features",
    labelCol="label",
    maxIter=50,        # number of boosting rounds
    maxDepth=5,        # depth per tree
    stepSize=0.1,      # learning rate
    seed=42
)
```



RF: Trees are independent (bagging) → easy to parallelize
GBT: Trees are **sequential** (boosting) → each corrects previous errors
GBT often wins on accuracy, RF is faster to train

GBT Limitation in Spark

Spark's GBTClassifier currently only supports **binary classification**. For multiclass, use Random Forest or Logistic Regression.

Instructor Notes

Gradient-Boosted Trees is the third and often the most accurate model. You may know it as XGBoost from your ML course — GBTClassifier is Spark's distributed equivalent. The key difference between RF and GBT is how trees are combined. In Random Forest, trees are independent and trained in parallel — this is called bagging. In GBT, trees are sequential — each new tree corrects the errors of all previous trees, which is called boosting. As a consequence, GBT often achieves higher accuracy but is slower to train because trees cannot be fully parallelized. We set `maxIter=50` for 50 boosting rounds, `maxDepth=5`, and `stepSize=0.1` as the learning rate. There is one important limitation: Spark's GBTClassifier only supports binary classification. For multiclass problems, you must use Random Forest or Logistic Regression.

Complete Pipeline — Putting It Together

```
from pyspark.ml import Pipeline

pipeline = Pipeline(stages=[
    crime_indexer,        # "Primary Type" -> crime_index
    domestic_indexer,     # "Domestic" -> domestic_index
    assembler,            # columns -> features vector
    rf                    # RandomForestClassifier
])

# Cache training data (iterative training benefits!)
train_df.cache()

# Train the entire pipeline
model = pipeline.fit(train_df)

# Predict on test data
predictions = model.transform(test_df)
predictions.select("features", "label",
                   "prediction", "probability").show(5)
```

Key Principle

`pipeline.fit(train_df)` fits **all stages** in order. `model.transform(test_df)` applies **all stages** in order. Compare with sklearn: `pipe.fit(X_train)` then `pipe.predict(X_test)` — same idea!

Instructor Notes

Now we put everything together into a complete pipeline. The stages array contains our four components in order: crime_indexer, domestic_indexer, assembler, and the Random Forest classifier. When we call `pipeline.fit(train_df)`, Spark fits each stage sequentially on the training data — indexers learn category mappings, the assembler is configured, and the classifier trains on the assembled features. Notice the `train_df.cache()` call before fitting. This is a performance optimization from Week 8 — ML algorithms are iterative, so the training data is read multiple times. Caching keeps it in memory to avoid redundant HDFS reads. After training, `model.transform(test_df)` applies every stage to the test data and produces predictions. This is the Spark equivalent of scikit-learn's `pipe.fit(X_train)` then `pipe.predict(X_test)`.

Outline

1. From Traditional ML to Distributed ML
2. The ML Pipeline API
3. Feature Engineering
4. Model Training
5. Model Evaluation
6. Model Comparison & Hyperparameter Tuning
7. Summary

What Predictions Look Like

features	label	prediction	probability	
[8, 0, 14, 1]	0	0.0	[0.82, 0.18]	✓
[11, 1, 22, 0]	1	1.0	[0.35, 0.65]	✓
[3, 2, 3, 1]	1	0.0	[0.55, 0.45]	✗

prediction

Model's binary answer: 0.0 or 1.0
(sklearn: `model.predict()`)

probability

Confidence: [P(class=0), P(class=1)]
(sklearn: `model.predict_proba()`)

Instructor Notes

Let us examine what the output of `model.transform()` looks like. Unlike scikit-learn where predictions are a separate array, Spark adds new columns to the DataFrame. You get three new columns: `prediction` is the binary answer — 0.0 or 1.0. `probability` is a vector with confidence scores for each class, equivalent to scikit-learn's `predict_proba()`. Look at the third row: the model predicted 0.0 but the true label was 1. The probability was [0.55, 0.45], meaning the model was only 55 percent confident — it was essentially guessing. This is why probability matters: it tells you not just what the model decided but how sure it was. In production, you might set a different threshold than 0.5 based on the cost of false positives versus false negatives.

Evaluation Metrics — You Know These!

Same metrics from your ML course. The formulas don't change in distributed ML.

Metric	Formula	Meaning for Our Task
Accuracy	$\frac{TP+TN}{\text{Total}}$	% of correct arrest predictions
Precision	$\frac{TP}{TP+FP}$	Of predicted arrests, how many were real?
Recall	$\frac{TP}{TP+FN}$	Of real arrests, how many did we catch?
F1 Score	$\frac{2 \cdot P \cdot R}{P+R}$	Balance between P and R
AUC-ROC	Area under ROC curve	Overall ranking quality (0.5 = random)

Why Not Just Accuracy? (Refresher)

Chicago crimes: ∼85% have No Arrest. A model predicting "No Arrest" for everything gets **85% accuracy** but catches **zero criminals**. Always check **F1** and **AUC**.

Distributed Metric Computation

Spark computes TP/TN/FP/FN counts **per partition**, then aggregates across workers. Same formulas, parallel counting.

Instructor Notes

These are the same evaluation metrics from your ML course — the formulas do not change in distributed ML. Let me remind you what each one means in our context. Accuracy is the percentage of correct predictions overall. Precision answers: of all the arrests we predicted, how many were real? Recall answers: of all the real arrests, how many did we catch? F1 is the harmonic mean of precision and recall. AUC-ROC measures overall ranking quality, where 0.5 means random guessing. The critical point is the class imbalance warning. With 85 percent of crimes having no arrest, accuracy alone is misleading. A model that always says "no arrest" gets 85 percent accuracy but zero recall. This is why we always report F1 and AUC alongside accuracy. Spark computes these metrics in parallel — it counts TP, TN, FP, FN per partition and then aggregates.

Confusion Matrix — Visual Guide

Actual \ Predicted	No Arrest (0)	Arrest (1)
No Arrest (0)	**TN** 6800	**FP** 350
Arrest (1)	**FN** 420	**TP** 1430

TN: Correctly said No Arrest ✔

FP: False alarm — wasted resources ✘

FN: Missed arrest — criminal walks free ✘

TP: Correctly predicted arrest ✔

Reading the Matrix

Precision $= \frac{1430}{1430+350} = 0.80$ Recall $= \frac{1430}{1430+420} = 0.77$ F1 $= \frac{2\times 0.80\times 0.77}{0.80+0.77} = 0.78$

Instructor Notes

The confusion matrix is the same visualization you know from your ML course, but let us interpret it in the context of crime prediction. The two green cells are correct predictions: TN means we correctly said no arrest, and TP means we correctly predicted an arrest. The two red cells are errors. FP is a false alarm — we predicted arrest but it did not happen, which wastes police resources. FN is a missed arrest — the crime led to arrest but our model missed it, meaning a criminal walks free. In this example, precision is 0.80, meaning 80 percent of our arrest predictions were correct. Recall is 0.77, meaning we caught 77 percent of actual arrests. The F1 score of 0.78 balances both. The question of which error is worse — FP or FN — depends on the application and is a domain decision.

Computing Metrics in Spark

In scikit-learn: `accuracy_score()`, `f1_score()`, etc. In Spark:

```
from pyspark.ml.evaluation import \
    BinaryClassificationEvaluator, \
    MulticlassClassificationEvaluator

# AUC-ROC
binary_eval = BinaryClassificationEvaluator(labelCol="label")
auc = binary_eval.evaluate(predictions)
print(f"AUC-ROC: {auc:.4f}")

# Accuracy, F1, Precision, Recall
mc_eval = MulticlassClassificationEvaluator(
    labelCol="label", predictionCol="prediction")

accuracy = mc_eval.evaluate(predictions,
    {mc_eval.metricName: "accuracy"})
f1 = mc_eval.evaluate(predictions,
    {mc_eval.metricName: "f1"})
precision = mc_eval.evaluate(predictions,
    {mc_eval.metricName: "weightedPrecision"})
recall = mc_eval.evaluate(predictions,
    {mc_eval.metricName: "weightedRecall"})
print(f"Accuracy: {accuracy:.4f}  F1: {f1:.4f}")
print(f"Precision: {precision:.4f}  Recall: {recall:.4f}")

# Confusion matrix
predictions.groupBy("label", "prediction") \
    .count().orderBy("label", "prediction").show()
```

Instructor Notes

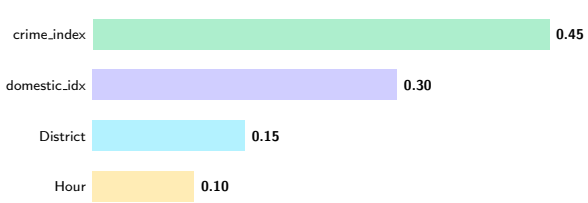
Let us look at the code for computing these metrics in Spark. There are two evaluator classes. `BinaryClassificationEvaluator` computes AUC-ROC, which is the default metric for binary classification. `MulticlassClassificationEvaluator` computes accuracy, F1, precision, and recall. Notice how you switch metrics by passing a parameter dictionary — this is slightly different from scikit-learn where each metric has its own function. For the confusion matrix, we use a simple `groupBy` on label and prediction columns and count the results. This gives you the same four cells as the visual matrix. It is important to note that these evaluators compute metrics in parallel across the cluster, so even on billions of predictions, the computation is fast. We will use these exact evaluators again when comparing our three models.

Feature Importances — Which Features Drive Predictions?

Same concept as sklearn's `model.feature_importances_`:

```
# Extract the fitted Random Forest model
rf_model = model.stages[-1]  # last pipeline stage
importances = rf_model.featureImportances

feature_names = ["District", "crime_index",
                 "Hour", "domestic_index"]
for name, imp in zip(feature_names, importances):
    print(f"  {name}: {imp:.4f}")
```



Interpretation — Matches Our Domain Reasoning!

Crime type is the strongest predictor (NARCOTICS → almost always arrest). **Domestic flag** second (mandatory arrest policies). This confirms our domain intuition from the feature selection slide.

Instructor Notes

Feature importances tell you which features the model relied on most for its decisions. This is the same concept as scikit-learn's `model.feature_importances_`. To access the Random Forest model within the pipeline, we use `model.stages[-1]` — the last stage is always the classifier. The results confirm our domain reasoning from earlier. Crime type has the highest importance at 0.45, which makes sense because narcotics crimes almost always lead to arrest while theft rarely does. Domestic flag is second at 0.30, consistent with mandatory arrest policies. District and Hour are moderate predictors. This is why we did domain reasoning before coding — it validates that our model is learning meaningful patterns, not just noise. If the feature importances contradicted our domain knowledge, that would be a red flag worth investigating.

Outline

1. From Traditional ML to Distributed ML
2. The ML Pipeline API
3. Feature Engineering
4. Model Training
5. Model Evaluation
6. Model Comparison & Hyperparameter Tuning
7. Summary

Logistic Regression vs Random Forest vs GBT

Aspect	Logistic Reg.	Random Forest	GBT
Training speed	✔ Fastest	Moderate	Slowest (sequential)
Accuracy (typical)	Baseline	Good	✔ Often best
Non-linear	✘ No	✔ Yes	✔ Yes
Interpretability	Coefficients	Feature importances	Feature importances
Overfitting risk	Low	Moderate	⚠ High
Parallelism	✔ Gradient parallel	✔ Trees parallel	⚠ Limited
sklearn equivalent	LogisticRegression	RandomForest	XGBoost / GBM

Recommended Strategy (Same as Traditional ML!)

Step 1: Logistic Regression (fast baseline). **Step 2**: Random Forest (handles non-linearity). **Step 3**: GBT (squeeze out extra accuracy).
If GBT $\gg$ LR, the relationship is non-linear. If LR $\approx$ RF, keep LR (simpler).

Instructor Notes

This comparison table summarizes the three models we have covered. Let me highlight the key tradeoffs. Logistic Regression is the fastest to train and has low overfitting risk, but it cannot capture non-linear relationships. Random Forest handles non-linearity and parallelizes well because trees are independent. GBT often achieves the highest accuracy but is the slowest because trees are sequential, and it has the highest overfitting risk. The recommended strategy is systematic. First, train Logistic Regression as a fast baseline. Second, train Random Forest to capture non-linearity. Third, train GBT to squeeze out extra accuracy. Then compare. If GBT significantly outperforms LR, the data has non-linear patterns. If LR performs similarly to RF, keep LR because it is simpler and more interpretable. This strategy is identical to what you learned in your ML course.

Hyperparameter Tuning — CrossValidator

In scikit-learn: `GridSearchCV`. In Spark: `CrossValidator` + `ParamGridBuilder`.
The big difference? Spark runs folds **in parallel across the cluster**.

```
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

# Define parameter grid (like sklearn's param_grid)
paramGrid = ParamGridBuilder() \
    .addGrid(rf.numTrees, [50, 100, 200]) \
    .addGrid(rf.maxDepth, [3, 5, 10]) \
    .build()  # 3 x 3 = 9 combinations

# CrossValidator (like sklearn's GridSearchCV)
cv = CrossValidator(
    estimator=pipeline,       # our full pipeline
    estimatorParamMaps=paramGrid,
    evaluator=BinaryClassificationEvaluator(labelCol="label"),
    numFolds=3,               # 3-fold cross-validation
    parallelism=4             # run 4 folds in parallel!
)

# Fit --- Spark tries all 9 combos x 3 folds = 27 models
cvModel = cv.fit(train_df)

# Best model is automatically selected
best_predictions = cvModel.transform(test_df)
```

The Distributed Advantage

`parallelism=4`: Spark trains 4 models **simultaneously** across the cluster. On a laptop, 27 models run sequentially. On a 4-worker cluster, ∼7× faster.

Instructor Notes

Hyperparameter tuning is where distributed ML really shines. In scikit-learn, `GridSearchCV` runs all parameter combinations sequentially on one machine. In Spark, `CrossValidator` runs them in parallel across the cluster. Let us walk through the code. First, we define a parameter grid with `ParamGridBuilder`: three values for numTrees and three for maxDepth, giving us nine combinations. With three-fold cross-validation, that is 27 models total. The critical parameter is `parallelism=4`, which tells Spark to train four models simultaneously. On a four-worker cluster, this gives roughly seven times speedup compared to sequential training. The `CrossValidator` takes our full pipeline as the estimator, so every transformation is correctly applied within each fold. After fitting, the best model is automatically selected based on AUC. We will experiment with this in detail in the hands-on session on Wednesday.

ML Best Practices Checklist

1. ✔ **Split before fitting**: Always split data *before* fitting any transformer (prevent data leakage)
2. ✔ **Check class balance**: Use stratified split or class weights if imbalanced
3. ✔ **Use Pipeline**: Ensures consistent transforms on train and test — **critical in distributed settings**
4. ✔ **Multiple metrics**: Evaluate with AUC, F1, precision, recall — not just accuracy
5. ✔ **Cache training data**: `train_df.cache()` before `.fit()` — ML algorithms are iterative, reads data many times
6. ✔ **Tune hyperparameters**: Use `CrossValidator` to find optimal settings — exploits cluster parallelism
7. ✔ **Save model**: `model.save("hdfs:///models/...")` for reuse without retraining

⚠ Data Leakage (Reminder)

If you fit a StringIndexer on the **full dataset** (before splitting), the test set "leaks" information into the model. Always fit inside a Pipeline that sees only `train_df`. *Same rule as scikit-learn — more dangerous at scale because bugs are harder to spot.*

Instructor Notes

Let me give you seven best practices that apply to every Spark ML project. First, always split before fitting to prevent data leakage. Second, check class balance — with 85 percent no-arrest in our dataset, this is critical. Third, always use a Pipeline to ensure consistent transformations. Fourth, evaluate with multiple metrics, not just accuracy. Fifth, cache your training data because ML algorithms are iterative and read data many times. Sixth, tune hyperparameters with CrossValidator to exploit cluster parallelism. Seventh, save your model to HDFS for reuse. The data leakage warning deserves special emphasis: if you fit a StringIndexer on the full dataset before splitting, the test set leaks information into the model. In distributed ML this is harder to spot because you cannot easily inspect intermediate DataFrames. Pipelines prevent this automatically.

Saving & Loading Models

In scikit-learn: `joblib.dump(model, "model.pkl")`. In Spark:

```
# Save the trained pipeline model to HDFS
model.save("hdfs:///models/arrest_predictor_rf")

# Later: Load without retraining
from pyspark.ml import PipelineModel
loaded = PipelineModel.load(
    "hdfs:///models/arrest_predictor_rf")

# Use immediately on new data
new_predictions = loaded.transform(new_data)
```

Production Workflow

1. Train pipeline once on cluster (expensive, maybe 30 min)
2. Save model to HDFS — accessible to **any machine** in the cluster
3. Load in production app (cheap, fast, milliseconds)
4. Retrain periodically with new data (weekly/monthly)

Why HDFS Matters

Unlike `joblib.dump()` which saves to **one machine**, Spark saves to **HDFS** — any node in the cluster can load the model. No file copying needed.

Instructor Notes

The last practical skill is saving and loading models. In scikit-learn you use `joblib.dump()` to save to a local file. In Spark you use `model.save()` to save to HDFS. The key advantage is that HDFS is accessible from any node in the cluster — you train on one machine and any other machine can load the model without copying files. The production workflow has four steps. First, train the pipeline once on the cluster, which may take 30 minutes. Second, save the fitted model to HDFS. Third, load it in your production application, which takes milliseconds. Fourth, retrain periodically as new data arrives. Notice that we use `PipelineModel.load()`, not `Pipeline.load()`. The fitted model includes all learned parameters — category mappings, feature weights, tree structures — so you can make predictions immediately without retraining.

Outline

1. From Traditional ML to Distributed ML
2. The ML Pipeline API
3. Feature Engineering
4. Model Training
5. Model Evaluation
6. Model Comparison & Hyperparameter Tuning
7. Summary

Key Takeaways

1. **Same math, different execution**: ML algorithms don't change — the **data distribution** and **parallel training** are what's new
2. **Spark MLlib** = distributed ML on DataFrames (`pyspark.ml`)
3. **Pipeline API**: Transformer → Estimator → Model — like sklearn pipelines, but essential for distributed consistency
4. **Feature engineering**: StringIndexer + VectorAssembler — same concept as LabelEncoder + column selection
5. **Three models**: LR (baseline) → RF (non-linear) → GBT (best accuracy)
6. **Evaluation**: AUC, F1, confusion matrix — same metrics, computed in parallel
7. **CrossValidator**: GridSearchCV equivalent — parallelized across cluster

The Big Picture

HDFS → MapReduce → Hive → Spark → **MLlib**

From *storing* data to *predicting* outcomes — the complete big data pipeline.

Instructor Notes

Let me summarize the seven key takeaways from today. First, the math does not change — what changes is the execution model with data distribution and parallel training. Second, Spark MLlib is the `pyspark.ml` package built on DataFrames. Third, the Pipeline API with Transformers and Estimators is essential for distributed consistency. Fourth, feature engineering uses StringIndexer and VectorAssembler, directly analogous to scikit-learn tools. Fifth, we covered three models in a systematic progression from LR baseline to RF to GBT. Sixth, evaluation uses the same metrics computed in parallel. Seventh, CrossValidator provides distributed hyperparameter tuning. The big picture is that we have now covered the entire pipeline: HDFS for storage, MapReduce for processing, Hive for querying, Spark for analytics, and now MLlib for prediction. Next week we add the final piece: streaming with Kafka.

What's Next

Wednesday — Hands-On Lab (Session 9B)

- Build the complete arrest prediction pipeline yourself
- Data prep → Feature engineering → RF → LR → GBT → Compare
- Experiment with CrossValidator and hyperparameter tuning

Week 10: Apache Kafka + Spark Structured Streaming

- Shift from **batch** to **streaming** processing
- Kafka architecture: Brokers, Topics, Producers, Consumers
- Spark Structured Streaming: real-time analytics on infinite data

The Journey So Far

Store (HDFS) → Process (MapReduce) → Query (Hive) → Analyze (Spark) → **Predict (MLlib)** → *Stream (Kafka, Week 10)*

Instructor Notes

On Wednesday you will build this entire pipeline yourself in the hands-on lab. You will start from raw data, engineer features, train all three models, compare their metrics, and experiment with CrossValidator. Everything we covered today in theory, you will implement in code. Looking ahead, Week 10 shifts from batch processing to streaming. We will learn Apache Kafka for message queuing and Spark Structured Streaming for real-time analytics. That will complete the entire big data pipeline: storing data with HDFS, processing with MapReduce, querying with Hive, analyzing with Spark, predicting with MLlib, and streaming with Kafka. Make sure you review today's slides before Wednesday so you can focus on implementation during the lab.